

Malleable software

Restoring user agency in a world of locked-down apps

The original promise of personal computing was a new kind of clay—a malleable material that users could reshape at will. Instead, we got appliances: built far away, sealed, unchangeable. When your tools don't work the way you need them to, you submit feedback and hope for the best. You're forced to adapt your workflow to fit your software, when it should be the other way around.

In this essay, we envision malleable software: tools that users can reshape with minimal friction to suit their unique needs. Modification becomes routine, not exceptional. Adaptation happens at the point of use, not through engineering teams at distant corporations.

Realizing this vision will require confronting many barriers. From programming languages to operating systems and app stores, every layer of the modern computing landscape has been built upon the assumption that users are passive recipients rather than active co-creators. What we need instead are computing systems that invite every user to gradually become a creator. Tools that compose into custom workflows, rather than siloed monolithic apps. Systems that support local groups in communal creation.

Fortunately, there's a lot of inspiration to draw on—both research work and commercial products spanning the past several decades. In this essay, we synthesize ideas from that body of work, as well as lessons learned from years of testing our own prototypes. Whether you're a software developer, a researcher, or any kind of computer user, we invite you to join us in striving towards a world of malleable software.



Please cite this work as:

Geoffrey Litt, Josh Horowitz, Peter van Hardenberg, and Todd Matthews. 2025. Malleable Software: Restoring User Agency in a World of Locked-Down Apps. Ink & Switch. <https://www.inkandswitch.com/essay/malleable-software/>.

We welcome your feedback: [@inkandswitch](https://twitter.com/inkandswitch) or hello@inkandswitch.com.



Ink & Switch

[Geoffrey Litt](#)

[Josh Horowitz](#)

[Peter van Hardenberg](#)

[Todd Matthews](#)

June 2025

[Print / View as PDF](#)

Contents

Motivation

- We want to adapt our environments
- Mass-produced software is too rigid
- Our goal: malleable software
- Existing approaches

A gentle slope from user to creator

Tools, not apps

- Apps are avocado slicers
- Sharing data between tools
- Composing the user interface

Communal creation

Ink & Switch prototypes

- Infrastructure for malleability
- Dynamic documents

Towards a better future

Motivation

We want to adapt our environments

Environments matter. To do our best work and live our best lives, we need spaces that let us each express our unique potential.

A guitar maker sets up their workshop with their saws, hammers, chisels and files arranged just so. They can also build new tools as needed to achieve the best result—a wooden block as a support, or a pair of pliers sanded down into the right shape.



Over the years, a home cook gradually assembles a combination of knives, cutting boards, and pots and pans, finding the ones that work best for them. They can install hooks on the ceiling and move shelves around to support their workflow—whether that’s cooking weekday dinners or hosting elaborate weekend cookouts.



These are everyday situations. **In the physical world, the act of crafting our environments comes naturally, because physical reality is malleable.**

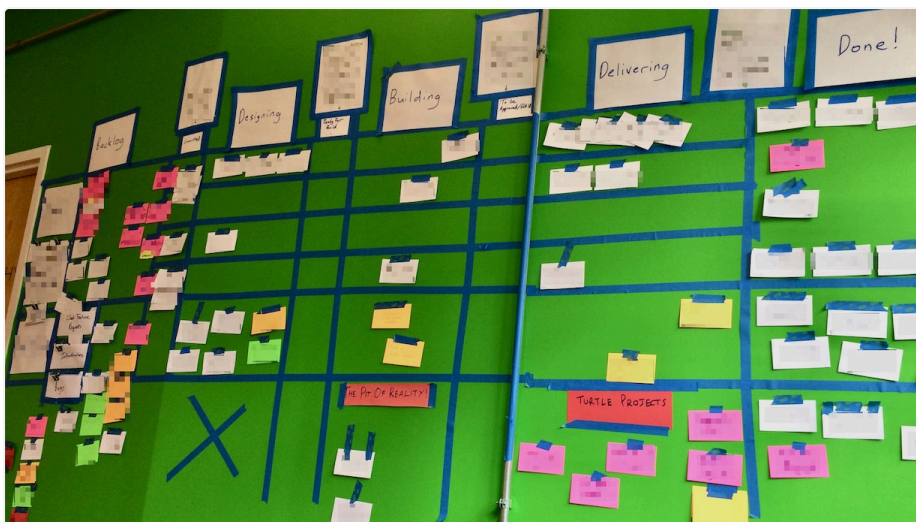
Many small tweaks—taping a post-it note to the wall, rearranging some drawers, moving a piece of furniture—can be done instantly without asking anyone’s permission. We can also take on larger changes that require more effort and skill, like building a workshop or renovating a kitchen. And should we lack those skills ourselves, we can recruit help from craftspeople in our local communities.

When we work and live in a physical space that we control, we tend to evolve it to suit our own needs. As Stewart Brand writes in his book How Buildings Learn: “Age plus adaptivity is what makes a building come to be loved. The building learns from its occupants, and they learn from it.”

Mass-produced software is too rigid

These days, we spend more and more of our time in environments built from code, not atoms. We’ve gained many capabilities in this shift—we can collaborate instantly across continents and search thousands of files in an instant. But **we’re also losing something important: the ability to adapt our environments and make them our own.**

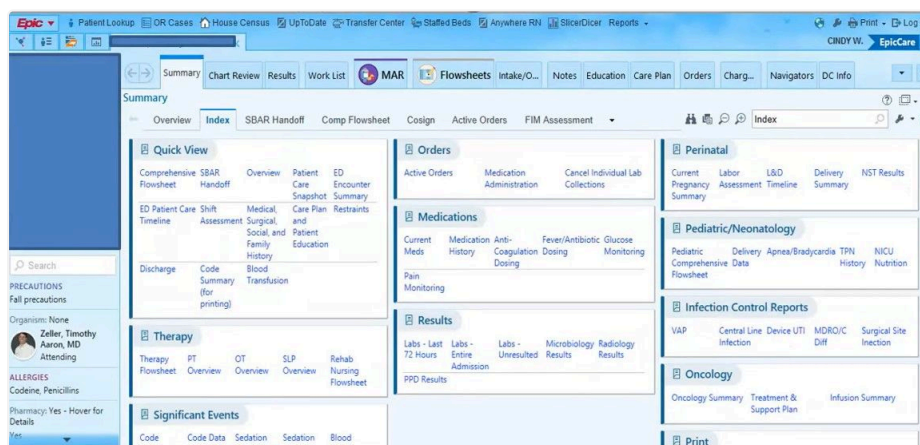
Here’s an example. One of the authors worked on a software team that tracked its work with index cards taped to a wall. The team would constantly evolve the tracker—tape lines moved; checklists appeared; special zones of cards emerged around the main grid. The fluidity of the tool encouraged fluidity of process.



A wall of index cards is a malleable medium for managing software development.

Later, the team switched to a web-based issue tracker to support remote collaborators. Now there wasn't a way to model or display a special zone of cards, so the team abandoned that part of their process. Further process changes also ground to a halt. Before, new ideas took minutes to try; now they could take hours of wrangling configurations, if they were possible at all. **Computerizing work led to a loss of agency.**

The rigidity of software isn't just a minor inconvenience. It can seriously impede people doing important work. The doctor and writer Atul Gawande has written about how computerization in the medical profession is leading to record levels of burnout. For instance, doctors would once skip irrelevant fields when filling out paper forms; now the software forces them to fill in those fields, and they have no power to edit those software rules. As Gawande says of one doctor: "Spending the extra time didn't anger her. The pointlessness of it did."



Inflexible electronic medical records systems are driving doctors to burnout.

When you face a situation where software doesn't meet your needs, you might try giving feedback to the developers—but that usually doesn't result in immediate action. **When different users have different needs, a centralized development team can't possibly address everyone's problems.** And for that matter, when a developer does try to cram too many solutions into a single product, the result is a bloated mess. To avoid this trap, good product teams learn to decline most user requests, leaving a long tail of niche needs unserved.

It may seem inevitable that our specific requirements aren't served by our software—but that's only because we've taken for granted that software is controlled by centralized development teams. What if we were to shift more control into the hands of users who know their own needs?

Gawande tells a story of a neurosurgeon who worked with an IT analyst to adapt his department's medical records system. "Before long, they had built a faster, more intu-

itive interface, designed specifically for neurosurgery office visits.” The requirements were driven by the needs of *their specific department*, not the needs of every doctor in the country. Beyond the direct productivity benefit, the physicians felt more in control of their tools—an antidote to burnout.

As inspiring as this story is, it’s more the exception than the rule, because **the tools and infrastructure we use to deploy software treat users as passive recipients rather than active co-creators**. Software is organized into monolithic applications rather than flexible remixable toolkits. Customization requires programming skills that most people don’t have—and besides, most software is closed source. Software doesn’t ship to users with the tools to edit the software. App stores are designed for companies distributing software to consumers, not amateurs sharing tools with their friends. This is a system of industrial mass production, not small-scale craft.

To be fair, mass-produced software has delivered many benefits. We can access a vast array of highly polished applications at reasonable prices. Software has made progress in reliability, accessibility, and security. Developers have created business models that can sustainably pay teams to deliver continuously improving software.

But as these stories and countless other examples show, inflexible mass-produced software also gets in the way. The more different you are from the average user, the more the benefits of customization outweigh the benefits of professional polish. Everyone is unique in some way—perhaps you have strong opinions about tools for writing, making music, having discussions, or planning projects. When you have specific needs, agency matters.

Our goal: malleable software

We envision a new kind of computing ecosystem that gives users agency as co-creators. We call this idea malleable software—a **software ecosystem where anyone can adapt their tools to their needs with minimal friction**.

The term “malleable software” has a long legacy and isn’t our invention. Recently it’s been invoked in research including Tchernavskij’s doctoral thesis and Klokmoose et al’s *Webstrates*. While the origins of the term are unclear, it may have begun with Alan Kay invoking the idea of software as “clay” in his 1984 article *Computer Software*.

- By “software ecosystem”, we mean the broad technical and cultural environment surrounding software and its users. Malleability isn’t a narrow technical problem.
- By “anyone”, we mean that broad accessibility is the goal. And while individual self-sufficiency is useful to cultivate, cooperating with local community is also valuable.
- When we say “adapting tools” we include a whole range of customizations, from making small tweaks to existing software, to deep renovations, to creating new tools that work well in coordination with existing ones. Adaptation doesn’t imply starting over from scratch.
- Finally, “minimal friction” is key. Editing our tools should be fast. It should feel light. At best, it should be something we can do in the moment when a need arises, so we can get back to the task at hand.

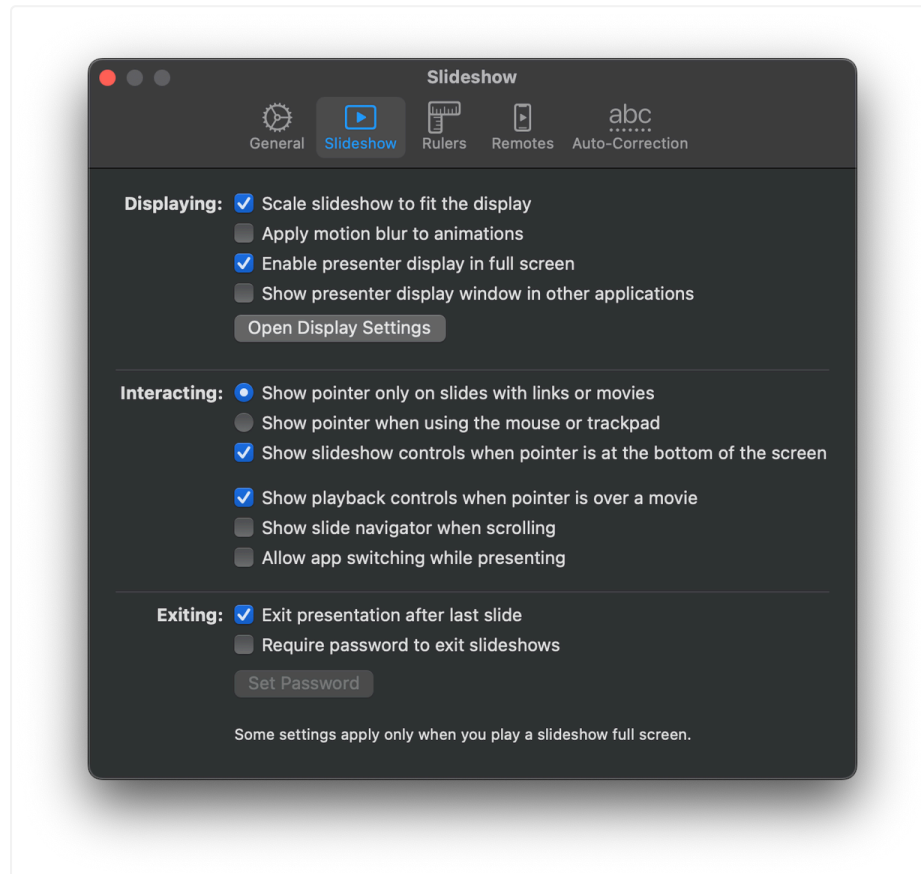
Existing approaches

You may be wondering: what about settings or plugins? There are indeed many techniques for customizing software that deserve to be celebrated. However, they also have limits that prevent them from fully achieving the goals we’ve laid out above.

Settings

Settings are a common way to change the way an application behaves. If the right setting exists, you can just toggle a checkbox and move on with your day.

But settings only offer controls that the application developers have thought to expose, leaving you stuck if there's not a setting that does what you want. Settings also tend to become long lists of disjointed checkboxes without a coherent mental model tying them together.



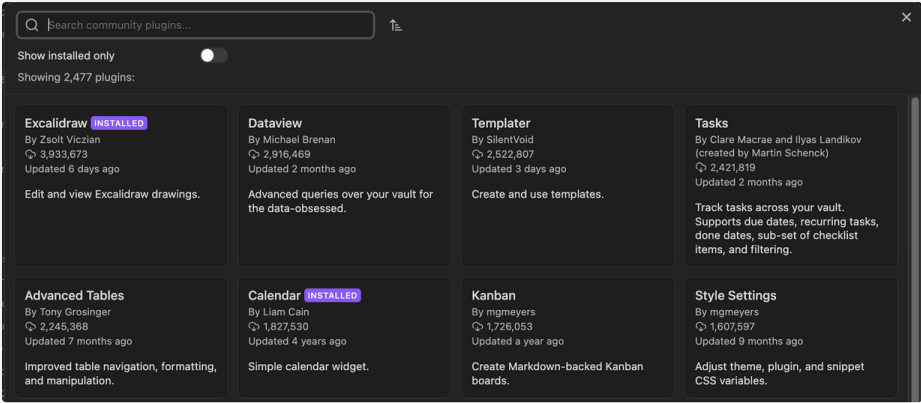
Settings allow for certain customizations chosen in advance by an application developer.

Plugins

One way to scale beyond the bandwidth of a central developer is to allow third-party plugins that extend the behavior of an application. A good plugin system makes it easy for users to get started customizing with a minimum of effort, because they can install plugins that other people have created. A plugin API also has the key benefit of stabilizing the contract between the underlying application and various extensions, helping with ongoing maintenance.

However, **plugin systems still can only edit an app's behavior in specific authorized ways**. If there's not a plugin surface available for a given customization, the user is out of luck. (In fact, most applications have no plugin API at all, because it's hard work to design a good one!)

There are other problems too. Going from installing plugins to *making* one is a chasm that's hard to cross. And each app has its own distinct plugin system, making it typically impossible to share plugins across different apps.



The Obsidian Markdown editor offers a rich ecosystem of community plugins for extending the editor's behavior.

Modding

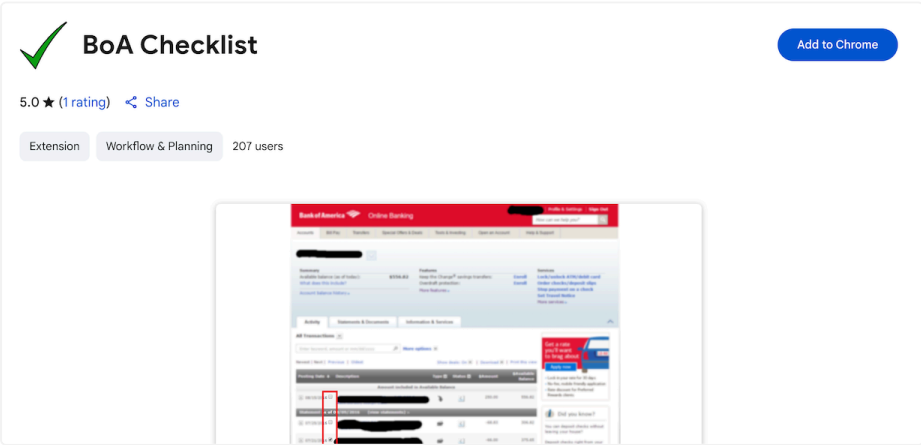
When settings and official extension APIs don’t go far enough (or aren’t provided in the first place), users can sometimes take control through permissionless *modding*. For instance, browser extensions can intervene in a website’s user interface and inject new client-side behavior without needing any hooks to be exposed by the original application developer.

The term “modding” seems to originate from video game modding. Examples of modding in other domains include the storied history of Mac kernel hacking and Prefab’s pixel-based reverse-engineering of interfaces. Modding overlaps considerably with the concept of “adversarial interoperability”.

Permissionless mods apply in a much broader range of scenarios than officially supported plugin APIs. In fact, they can even be used when the original developer is actively opposed to a given type of extension—ad blockers prioritize the interests of users over the interests of websites.

Mods face their own limitations. They can require tedious reverse-engineering to create. They are often difficult to maintain as the underlying application evolves. Different mods on the same app often don’t work together cleanly.

The limits of the underlying platform can also limit what they’re able to do—for example, **browser extensions can’t modify server-side behavior**, severely limiting the features they can provide. Also, much like other kinds of plugins, unsupported mods have a wide chasm between *installing* and *creating*. For instance, to make a browser extension, you need to leave the browser, write code, and distribute a packaged code artifact. (Later on we’ll discuss some strategies to avoid these steps.)

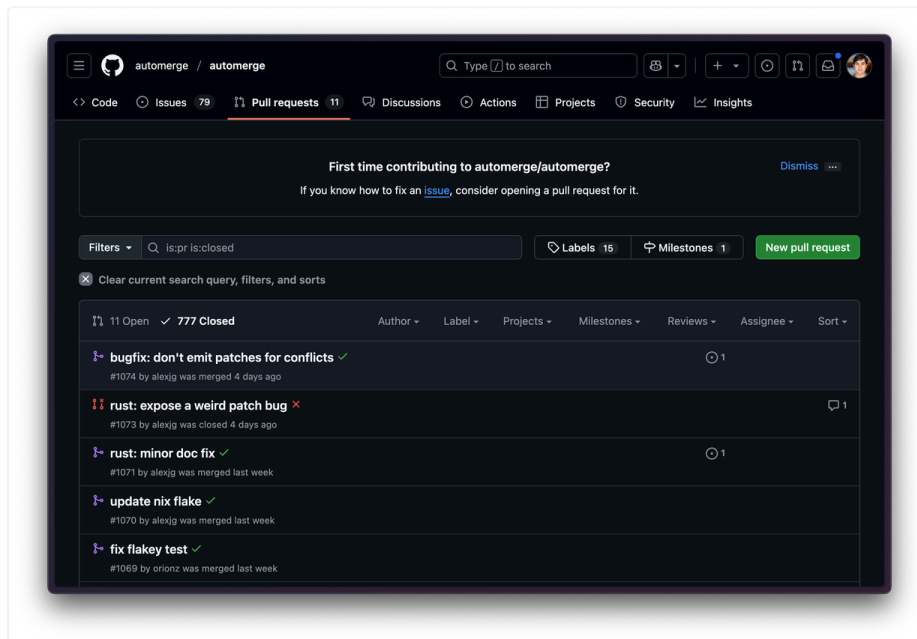


An example of a browser extension serving a niche need: BoA Checklist adds a persistent checkbox next to each transaction on the Bank of America website, to track if you've already reviewed that transaction.

Open source

The open source software movement promotes the idea of distributing source code so that users of software can own their software, contribute back to it, and if needed create their own version that better meets their own needs. This has been a positive force for the world, and represents an important ingredient for malleability.

But having access to edit the code doesn't mean minimal friction. **Modifying a serious open source codebase usually requires significant expertise and effort.** This applies even for making a tiny change, like changing the color of a button. Even for a skilled programmer, setting up a development environment and getting acquainted with a codebase represents enough of a hurdle that it's not casually pursued in the moment.



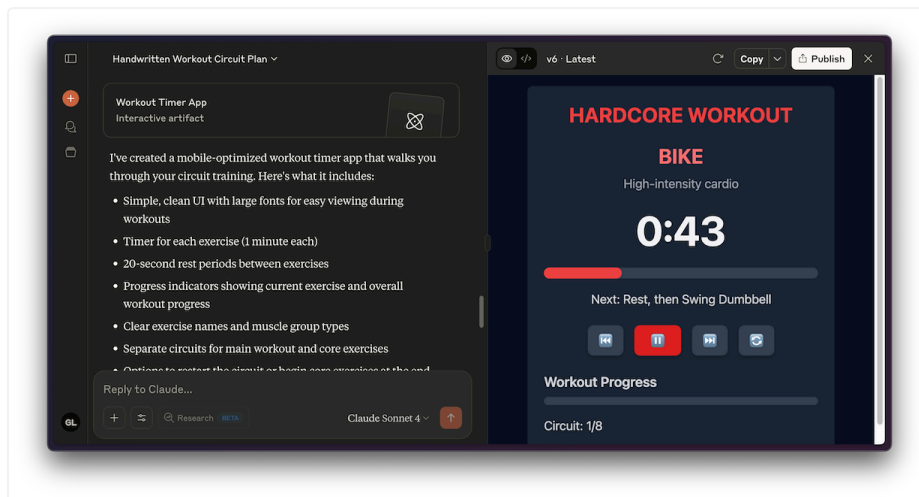
Users can propose changes to an open source project on GitHub.

AI-assisted coding

Finally, it's worth addressing a hot topic of the moment. Will AI tools automatically make malleable software a reality?

Historically, one of the biggest puzzles in democratizing the creation of software was figuring out how to enable regular people to write computer programs. Many approaches have been tried: programming languages with friendlier syntax or simpler semantics; programming-by-demonstration macro recording systems; visual live programming environments, and more.

Today, large language models offer a new approach: to take fuzzy ideas expressed in natural language and automatically turn them into code. There is now momentum towards a world where anyone can generate a web application from a chat, without needing any programming experience. There are new examples every day, from a journalist making an app for generating school lunch ideas to an environmental studies student making a website for tracking reforestation efforts. We've personally found it useful to build our own software tools with AI, such as a specialized Japanese translation app, and a minimal workout timer with a custom workout plan:



Building a workout timer tool using AI in Claude Artifacts

We think these developments hold exciting potential, and represent a good reason to pursue malleable software at this moment. But at the same time, **AI code generation alone does not address all the barriers to malleability**. Even if we presume that every computer user could perfectly write and edit code, that still leaves open some big questions.

How can users tweak the *existing* tools they've installed, rather than just making new siloed applications? How can AI-generated tools compose with one another to build up larger workflows over shared data? And how can we let users take more direct, precise control over tweaking their software, without needing to resort to AI coding for even the tiniest change? None of these questions are addressed by products that generate a cloud-hosted application from a prompt.

Bringing AI coding tools into today's software ecosystem is like bringing a talented sous chef to a food court. If you're used to purchasing meals from a menu, a skilled chef can't do much to help you. Similarly, if you're using closed-source software from an app store, an AI coding assistant can't do very much to help you as a user. To fully take advantage of the capabilities of AI, we need to move past the food court to something more like a kitchen—a site of open-ended creation.

Reshaping the foundations of computing around user agency is an ambitious goal. Luckily, since the dawn of personal computing, many people have proposed powerful ideas for achieving it.

In the next few sections, we'll summarize three design patterns that we see as essential for achieving malleable software. These aren't precise recipes for how to solve the problem. But we believe they are important ways of thinking that are underrepresented in today's software world and will form part of the solution to achieving malleable software.

This essay isn't an exhaustive survey of all work on malleability. For a longer reading list, we recommend the [Malleable Systems Collective catalog](#).

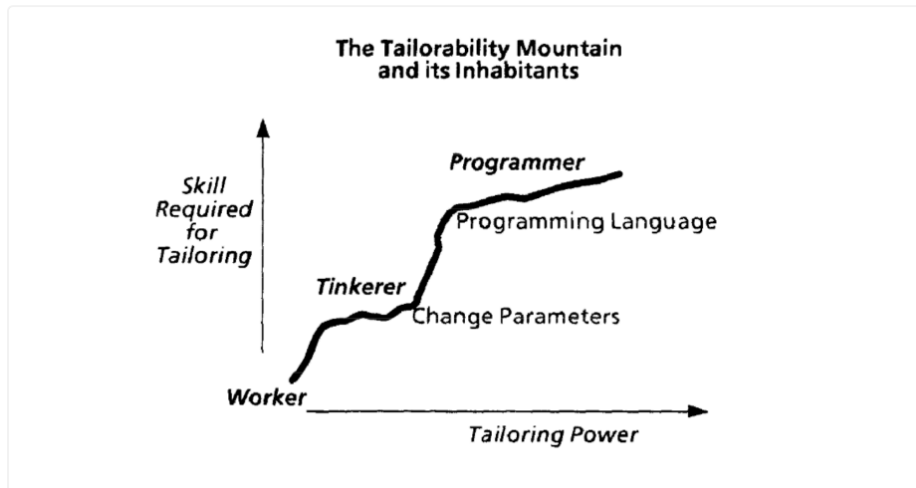
A gentle slope from user to creator

Malleable software does not imply everybody creating all of their own tools from scratch. That would be a waste of time, since many off-the-shelf tools are already pretty close to serving our needs, with only minor tweaks needed.

A more sensible approach is to start out using existing software tools built by other people or companies, but to have the *option* of modifying these tools as you discover ways that they don't meet your needs. You can start out as a passive user, and *gradually* become an editor and a creator.

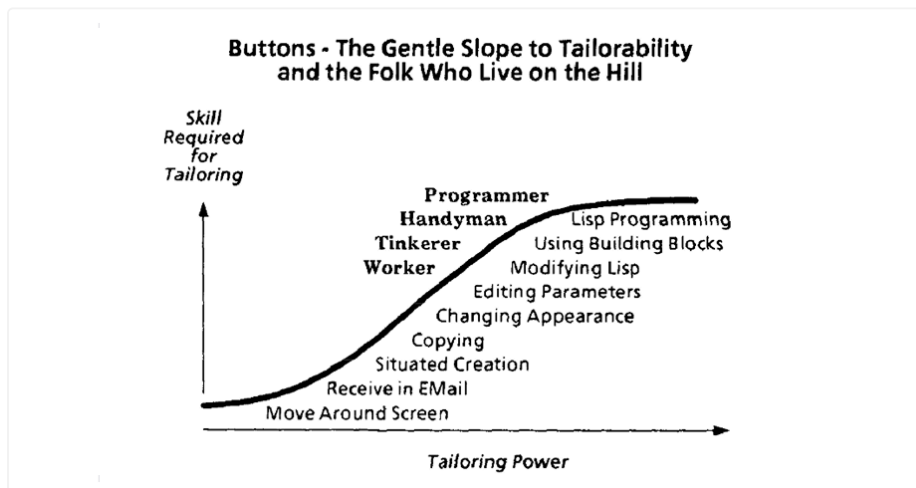
In a 1990 paper User-Tailorable Systems: Pressing the Issues with Buttons, Allan MacLean and his collaborators at EuroPARC propose a powerful mental model for designing software systems that invite users to gradually become creators.

Consider this chart where the x-axis represents the power or depth of a customization, and the y-axis represents the level of skill required to perform that customization. In this model, whenever someone needs to suddenly learn a lot more in order to achieve the next level of depth of customization, that appears as a vertical "cliff".



Going from changing settings to full programming is a sharp "cliff" that's hard to scale. (figure from MacLean et al)

To flatten out the cliffs and encourage smooth progress, MacLean et al. proposed a design goal: **each incremental increase in tailoring power should only require a small incremental investment of learning and skill.** A system that follows this rule can be visualized as having a "gentle slope" free of cliffs.

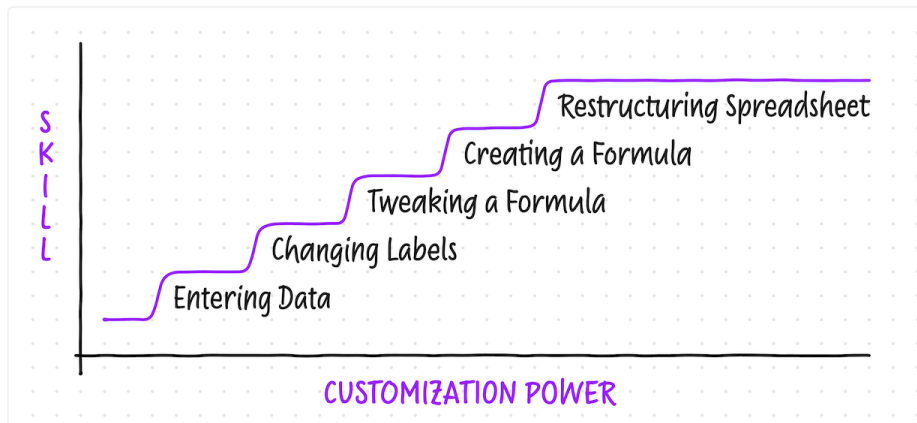


A gentle slope of tailorability, made up of many different customization techniques. (figure from MacLean et al)

The researchers demonstrated this principle in an automation system called Buttons. To start out, users could move buttons around and change the text or color. Slightly further up the slope, users could edit variable values or use a toolkit to create simple UIs. Finally, at the top of the slope, users could create new behaviors by doing Lisp programming. The key point was that **each customization could be done with the simplest technique possible, leaving full programming only as a last resort when absolutely needed.**

Alan Kay has described a similar philosophy of empowering users to “open the hood”.

Many successful environments for end-user malleability employ the gentle slope pattern. Consider spreadsheets as an example. When someone sends you a complex spreadsheet, you can start by just viewing it, or editing a cell marked as an input. Next you might try changing some formatting or add some labels. As you get deeper into using the spreadsheet, you might start tweaking formulas or adding your own new cells. A spreadsheet can start out feeling like an “application”—an artifact that someone else put together, with a few knobs intended for you to control—but unlike an application, you can smoothly progress to deeper customizations.



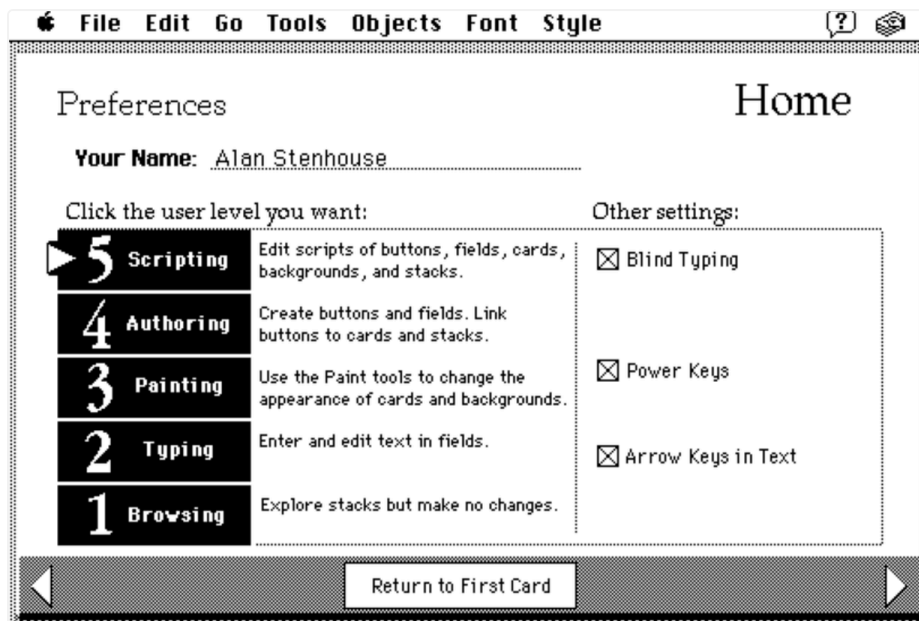
Spreadsheets offer a gentle slope of customization, from use to deeper modification.

Spreadsheets show us a couple useful qualities for achieving a gentle slope. First, spreadsheets have an in-place toolchain: everyone using a spreadsheet is running the full editor. When you want to dive deeper into modifying a spreadsheet, there’s no need to install or open a separate development environment.

Also, spreadsheets often have a scrappy aesthetic. It can *feel* safer to change a spreadsheet, compared to the pixel-perfect polish of a professionally designed application.

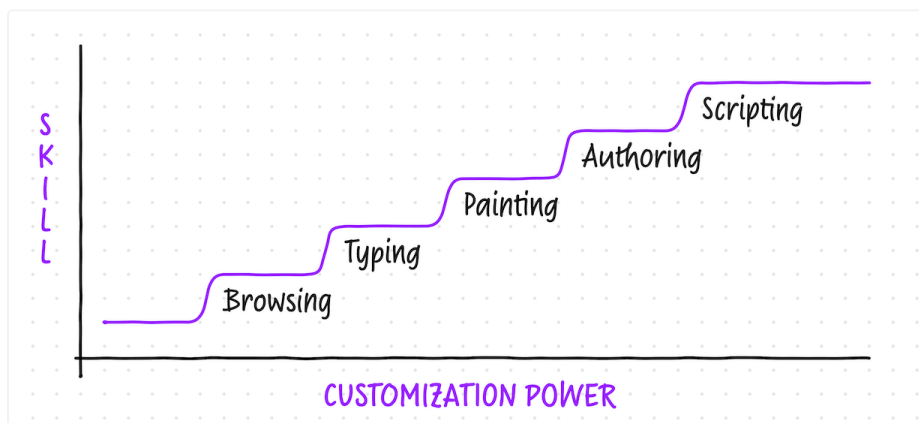
In *How Buildings Learn*, Stewart Brand reflects on the adaptability of shabby buildings: “Any change is likely to be an improvement... Do what you want. The place can’t get much worse anyway.”

Another way to achieve a gentle slope is through explicit modes. HyperCard, a Mac program from the 1980s in which users could create stacks of “cards”, offered “levels” which governed what actions were available to the user at any time.



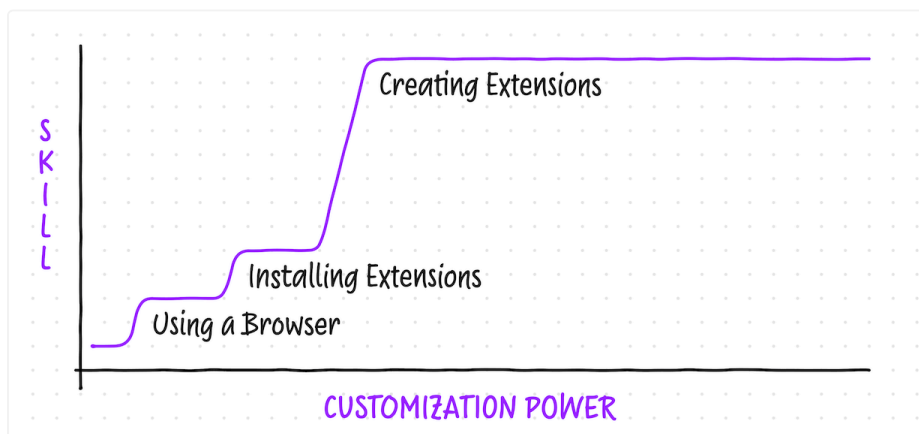
In HyperCard, users could select a "level" to restrict their editing capabilities.

Level 1 was read-only; Levels 2 and 3 supported text and graphical editing with direct manipulation interactions; Level 4 added creating buttons and linking things together; Level 5 unlocked full programming using a scripting language called HyperTalk. These explicit modes served as guardrails, allowing users to safely explore without unintended consequences.



Each level in HyperCard represents a modest step up in power and skill.

Finally, let's see what it looks like to smooth out a cliff. As we discussed earlier, there's a big chasm from installing a browser extension to building one yourself.



Going from installing a browser extension to making one is a steep cliff. It requires leaving the browser and doing involved programming.

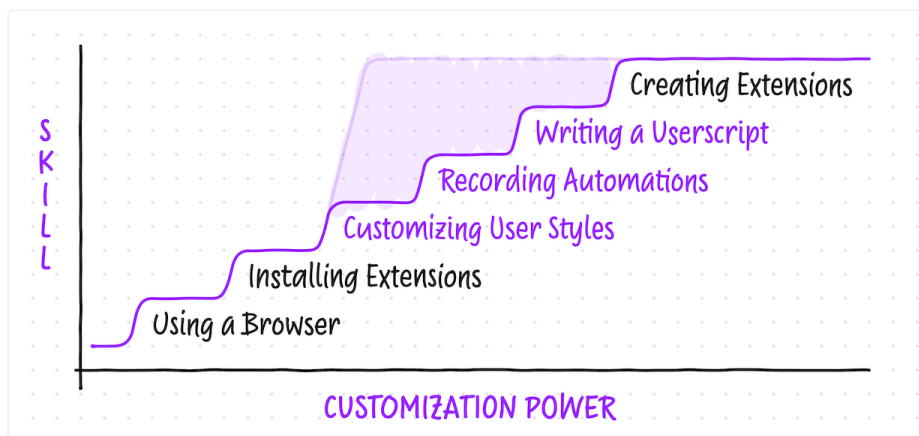
We can achieve a much smoother slope if we incorporate lighter weight techniques for customizing websites. Tools like Stylish and Arc Boosts let users create and share

custom styling themes from within the browser. Further up the curve, macro systems and let users record and replay actions taken on websites. And if the user knows a little bit of JavaScript, they can tweak user scripts written in Tampermonkey directly in the browser.



Arc Boosts lets users restyle a website with direct manipulation interactions from directly within the browser.

Each of these techniques gives a different tradeoff between power and skill. If someone just wants to tweak a color on a website, they should be able to do it in a less expressive system focused on that task, rather than needing to build an entire browser extension. With these techniques, we can smooth out the slope:



Web customization techniques provide a gentler slope before needing to create a browser extension.

There are two ways to make a slope gentler. One way is to start from the right side of the diagram and reduce the required skill. This strategy corresponds to a huge variety of work which starts with the full expressiveness of programming and makes it more approachable—with friendlier syntax, live programming environments, inferencing programs from examples, gradual scaffolding and more.

However, there's another strategy that's just as important, and sometimes overlooked: starting from the left side of the diagram and gradually offering more power. Many successful customizable systems such as spreadsheets, HyperCard, Flash, Notion, and Airtable follow a similar pattern: a **media editor with optional programmability**. When an environment offers document editing with familiar direct manipulation interactions, users can get a lot done without needing to write any code.

One final note: **not everyone needs to reach the top of the slope**. At some point, many amateurs will enlist help from others in order to save time and energy, or to avoid having to learn the necessary skills. The key is that many customizations are accessible to amateurs, with reasonable incremental investments of learning and skill. A gentle slope also supports smoother collaboration between people of different skill

levels. We'll explore this point more later on when we discuss the importance of communal creation.

Tools, not apps

So far we've focused on the rigidity of individual applications. But there's another reason that it's hard to adapt software to meet our needs: the very idea of "applications". Because we're so accustomed to this idea, it can be hard to see the nature of the problem—so let's consider an analogy from the kitchen.

Apps are avocado slicers

One way to slice an avocado is to use an "avocado slicer": a 3-in-1 gadget that combines a dull plastic knife for slicing the avocado in half, a circular grabber for extracting the pit, and a line of plastic rods that produce 7 slices at once.



An avocado slicer is specialized to one use case.

Anyone can use an avocado slicer with no practice, and it poses no safety risk. And yet, because the avocado slicer is narrowly focused on one task, it's useless at anything else. If you used a specialized gadget for every single task, you'd end up with a mountain of plastic.



There are specialized gadgets available for an enormous variety of tasks, from stripping corn to cutting lettuce.

Another approach is to use a knife. A knife can handle all the steps of slicing an avocado, and much more: it can slice a chicken breast, dice an onion, or smash a garlic clove. You do need to learn how to handle the knife safely and skillfully, but it's worth the effort, because **a knife is a general tool**.



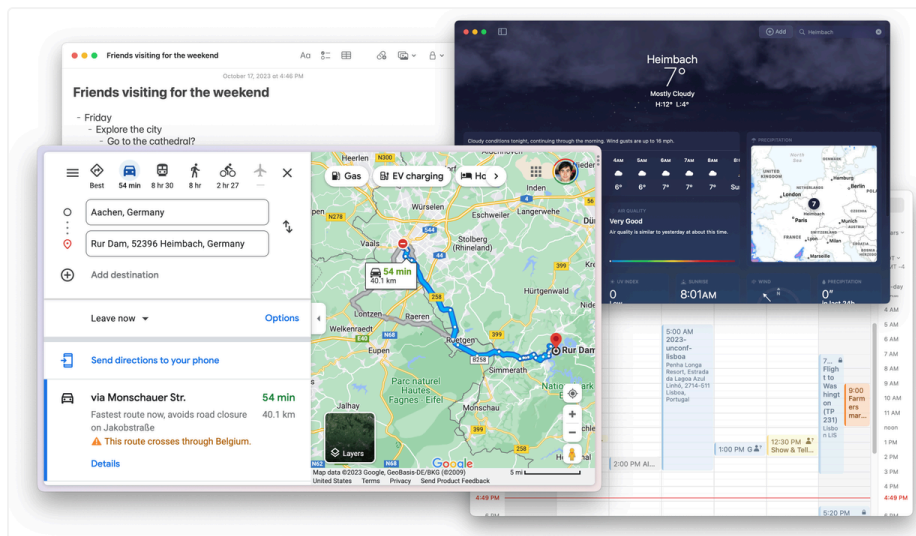
A knife can cut an avocado, and much more.

How does this analogy apply to software? **Many applications are avocado slicers.** They're a bundle of functionality targeted at some specific use case: planning a trip, tracking workouts, organizing recipes. Because an app needs to handle many tasks associated with a use case, it sometimes doesn't handle any of them particularly well. You may have come across situations where an app is missing some functionality that's important to you, while simultaneously including extra bits you don't need.

There are some notable exceptions to "apps are avocado slicers". Text editors and spreadsheets, for example, have knife-like qualities. People spend years mastering these tools and then can apply them to diverse tasks.

On top of that, solving a larger task using multiple applications often requires manual coordination. We can put windows next to each other and copy-paste data, but

not much more. If we want more knife-like software tools, we'll need better ways for smaller software tools to work together.



Planning a vacation requires manually coordinating across many isolated applications.

This isn't to say that the application model has no benefits. Users get a cohesive experience within the bounds of an application, since the developer can finetune it without worrying about interoperating with other tools. Siloing data by application is a convenient answer to some security and privacy concerns. Paying for an application (and support) is a reasonable billing model. These are important benefits which can't be ignored.

But when you're trying to solve a complex problem or construct a creative workflow spanning multiple applications, the isolation between applications can cause serious problems. **How might we reorient software around more general, composable tools**—that feels more like a knife and less like an avocado slicer? There are two sub-problems to address: sharing data between tools, and combining tools within the user interface.

A key inspiration in this area is Michel Beaudouin-Lafon's work on [Instrumental Interaction](#) described in his talk [A World Without Apps](#).

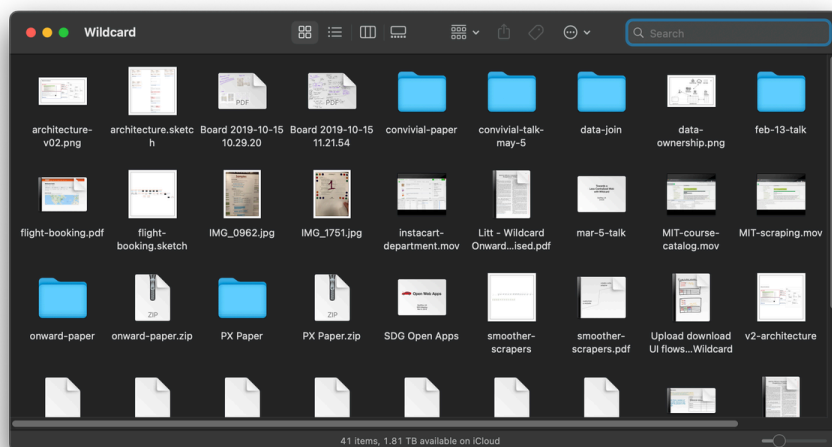
Sharing data between tools

If we're going to use different software tools together in coordination to get a job done, it's essential that those tools can operate on one shared reality.

In modern cloud and mobile platforms, each application manages its own data in a private silo. The plan for the trip lives across a notes app, a Google Maps list, a calendar. This fragmentation impedes malleability. **When each application manages its own data, introducing a new tool into a workflow incurs more friction.** It's easier to stay within an existing application to complete the whole job, even if superior alternatives exist for completing some part of the task.

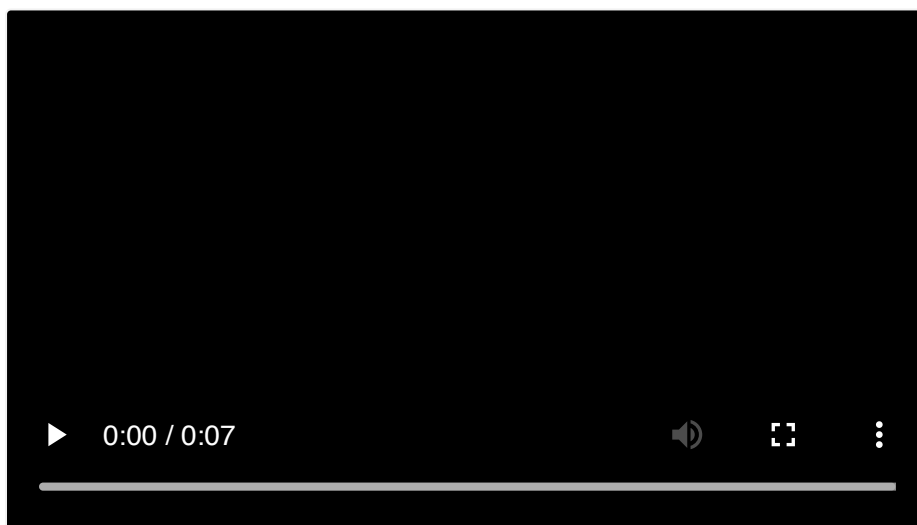
When data is instead shared among applications, it empowers end users to compose tools in more flexible ways. One well-known example is the desktop filesystem. When information is stored in files, **you can edit the same file in different tools that focus on different subtasks.** You can edit the light levels on a photograph in Lightroom, and then use Apple Preview to scribble a note on the photo. **Different collaborators can also use their preferred tools**—one user can edit a code repository in VSCode while collaborating with a coworker who edits it in emacs.

We believe a missing ingredient in today's ecosystem is a modern equivalent to the filesystem, with better support for collaboration. Later we'll share our prototypes of this idea.



Files serve as a common ground between different tools.

Another example of tools operating on shared data is low-code database application builders—including desktop-based applications like Microsoft Access and Filemaker Pro, and cloud-based applications like Airtable and Notion. These environments support end-users in building multiple views of the same database. For example, Airtable comes with with general tools such as a grid table editor, a board of cards, and a calendar. It also offers tools for adding custom UIs, but those are only needed in specialized cases, since many tasks can be achieved in the general grid editor.



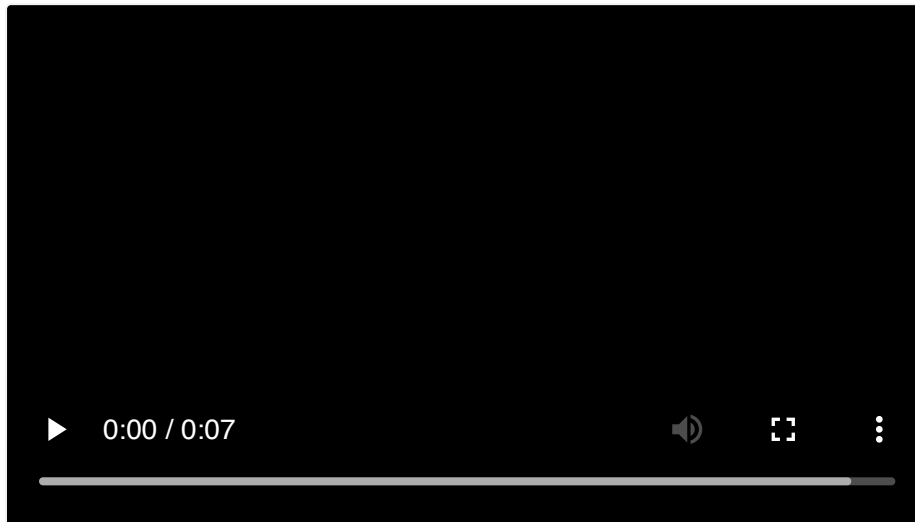
Editing the same data with multiple views in Airtable, with live reactivity

The idea of shared data also extends to shared objects with behavior. In Smalltalk, a user works with an *image*: a repository of objects representing not only persisted state, but also code associated with that state. Because everything in the system is represented as objects, it's possible to introduce new tools that operate on those existing objects—including calling methods to invoke behavior.

Inert data and rich objects each have advantages. Objects can help tools interpret and work with data in meaningful ways; data can be more straightforward to manage.

Finally, **realtime collaboration over shared data enables different people to work live in different tools**. One example of this is in Webstrates, a malleable software plat-

form for collaboration in the browser. In Webstrates, tools run by different users can collaboratively edit the contents of a shared, synchronized dataset stored in the browser's DOM. As a result, two users can each use their preferred editor to work on the same research paper together, with one of them using a WYSIWYG editor and the other using a plain text interface.



The same paper being edited realtime in two different editors, in Webstrates

Composing the user interface

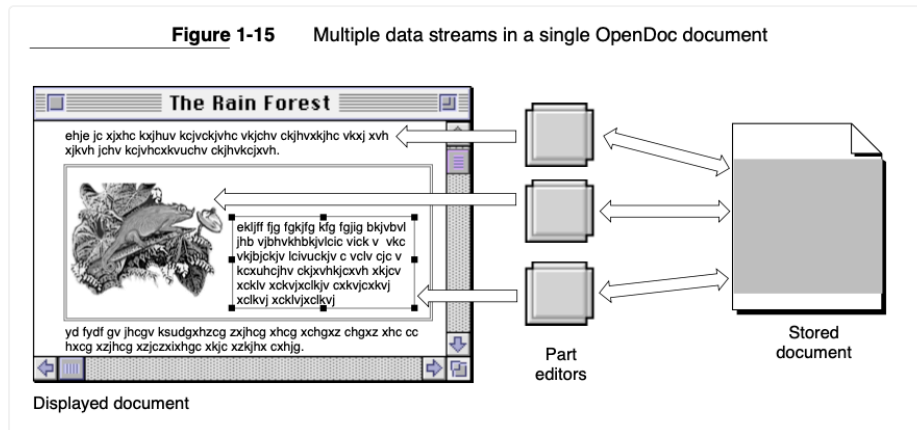


Tools need to share not only underlying data, but a workspace where they can be used together.

The app paradigm isn't set up for this. An application takes control over a window or tab and builds a world inside of it. This has advantages – an application's creator can carefully craft your experience in this world and make sure everything works together just right. But it means that **using multiple applications together requires flipping between isolated user experiences**. It's as though you have to carry ingredients from one kitchen to another every time you want to use a new tool. Even if data interoperates, your experience is splintered.

How can we break up not just an application's ownership of data, but its control of the interactive environment? Earlier, we discussed approaches like plugins and permissionless modding, which crack open seams to extend applications from within. These are great, but if we want a world where we work with “tools, not apps”, we need something a little different – environments where tools can be brought together and used in a shared space.

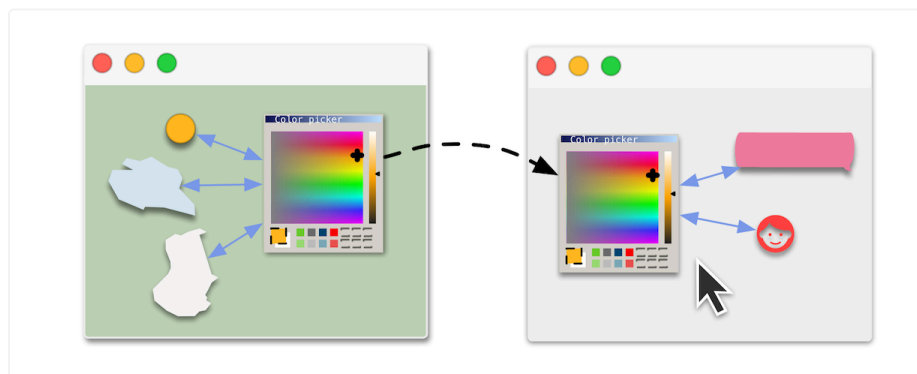
One set of ideas comes from **compound-document systems**, like OpenDoc and OLE. These let authors compose documents out of multiple kinds of media embedded in each other. For instance, a text document might host an embedded diagram, which can in turn host a table of data. Selecting one of these parts opens up an editor in-place, with no flipping back and forth between windows or digging through the file system required. These systems are open-ended; developers can introduce new kinds of media, as well as new, alternative editors for existing media types.



OpenDoc documents are made of "parts" that are edited with different editors, but stored together in a unified document.

Splitting up the UI into smaller pieces raises new design challenges. How do users choose which editors to use for each part of a document? How can a user share a document and guarantee that others will have all the necessary sub-editors available? How do global interactions like selection and focus work across editors? Systems like OpenDoc developed answers to these questions which are worth studying, although the nature of modern collaborative and web-based applications have also changed the surrounding context.

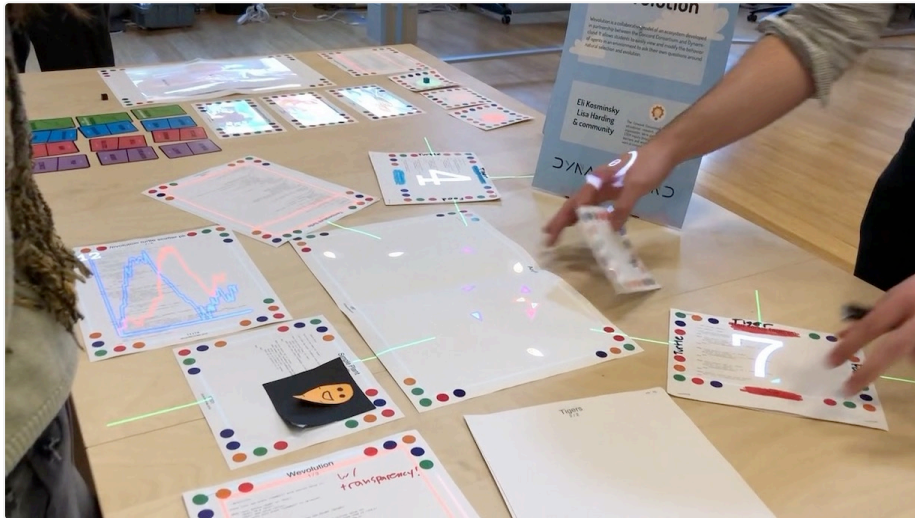
Sometimes, embedding editors for parts of a document is not enough. Consider this example from Tchernavskij's Designing and programming malleable software: **how might a user reuse a color picker from one app inside of another app?** Unlike a diagram editor, a color picker is not associated with just one part of a document; it's a tool that interacts with objects *throughout* the document. In order to do its job, the color picker somehow needs to be aware of other objects and have the ability to edit their colors. Usually, user interfaces hardcode these kinds of connections, but to allow extension or reuse, we need a more flexible and open-ended approach.



A challenge problem from Designing and programming malleable software: reusing a color picker from one application in another.

To solve this, Tchernavskij proposes *entanglers*: a dedicated layer of the UI system that dynamically detects and connects related UI elements. Another research project, Varv, solves the problem by specifying behavior with lists of event triggers which can be additively grown. While the details of these approaches differ, the common element is representing connections between parts of the UI in such a way that supports later extension.

A more radical approach to UI composition is to use the physical world as a foundation for composability, as seen in Dynamicland. Programs at Dynamicland are built as collections of physical objects (like pieces of paper) that can carry code and data and act as handles for interaction. At Dynamicland, you can compose user interfaces by bringing them near each other on a table and taping or gluing them together. Programs can also use relationships in space, like pointing and proximity, to create virtual connections between objects. While screen-based systems can't directly emulate Dynamicland's use of the physical world, they may still be able to take some lessons from the value of having a consistent materiality and physics.



Pieces of paper running programs at Dynamicland, assembled on a table, sending out green "whiskers" to define their connections.

Dynamicland also derives much of its composability from how it breaks up programs into small objects that work together. Communication between these objects happens through a global database. As a result, it's possible to add new tools into the system that intercept messages between existing objects, modifying the behavior of existing tools. When a system doesn't assume up front that certain connections exist between senders and receivers of information, it's easier to additively extend the system with new uses of that information.

Systems including Smalltalk, Boxer, and Webstrates take a similar approach to Dynamicland, combining "content, computation and interaction" into an integrated, end-user programmable "substrate". This term was established by Webstrates and continues to be developed, including by a recent workshop.

Communal creation

While it's exciting to imagine every person crafting their own unique computing experience, an individualistic view of malleable software will only get you so far.

Sure, individuals should be able to adapt their software to their needs in small ways in the moment, climbing up a "gentle slope from use to creation". But as the changes they want to make get larger, software is going to take more time and skills to make. If everyone were forced to make these changes just for themselves, as one-offs, the benefits would probably not be worth the costs. Not everyone wants to be a software-making expert.

And there's an even more fundamental reason to think of malleability through a communal lens: we use computers together! A product team needs a single system for tracking projects. A department at a hospital needs a single system for patient intake forms. These communities are certainly not well-served by one-size-fits-all applications they have no control over. But the solution can't be every-user-for-them-

selves either. We should help communities build and maintain *shared* solutions to their problems.

See [The inequities of our remote gathering tools](#) for more on how one-size-fits-all apps like Zoom can disrupt democratic social dynamics.

With the right infrastructure, we can work together to craft our software. People with similar needs around the world can exchange work and build collaboratively, as we have seen happen in free-software communities. And local communities, from companies to families to civic organizations, can build and maintain software suited to their local needs. When local needs get higher up the “slope” and call for special levels of skill and enthusiasm, you don’t need everyone in a community to attain these levels – just enough people to play that role and get the job done.

Studying spreadsheet use at companies, Nardi and Miller found “local developers” – amateur enthusiasts who could write bits of code for their coworkers and guide their team up the gentle slope of spreadsheets.

Building software for “local” contexts is sometimes *easier* than building software for world-wide use. You don’t need to build airtight, industrial-grade software if you are in direct contact with your users and can be responsive to the situations they run into. You don’t need to anticipate everyone’s needs in your design, just your community’s. Clay Shirky termed this pattern “*situated software*”, describing how his students were able to rapidly build software for their communities by “taking advantage of social infrastructure or context-sensitive information”. On an even more intimate level, Robin Sloan memorably described how an app built for his family could be a “*home-cooked meal*”.

For all these reasons, we believe that *technical* infrastructures for malleable software will need to support *sociotechnical* systems of people working together, across many levels, to make software work for themselves and their communities.

The history of free software offers lessons on how sociotechnical systems like this can be constructed. We are especially inspired by situations where free-software communities don’t assume there should only be one centrally-controlled version of an application in the world. For instance, Mastodon instances run by small communities often [edit Mastodon’s code](#) to implement community-specific features and policies. Of course, situations like this still require serious engineering work, and are still operating on apps. As we move towards a world with gentler slopes into software modification, and more tools rather than apps, we’ll need to figure out how smaller pieces of code can be shared and collaboratively developed, and how interoperability can be maintained in a world of pluralistic software.

Ink & Switch prototypes

At Ink & Switch, we’ve spent several years building research prototypes that explore different aspects of malleable software. These projects aren’t commercial products. Rather, the goal of each prototype has been to develop our understanding of techniques for enabling malleability, and then to learn from deep usage of those prototypes. (In fact, we wrote this very essay in a homegrown malleable software environment, and will share some of our successes and failures from that experience.)

We’ve previously published essays about most of the projects mentioned in this section; follow the links for more detail.

Our work has spanned the entire computing stack, and can be roughly grouped into two threads. One thread has explored **foundational infrastructure**—techniques for storing data, loading code, and defining user interfaces, in a way that supports malleable experiences being built on top. Higher up the stack, another thread of work has focused on a particular kind of user experience: **dynamic documents** where static media can be gradually enriched with interactive behavior. Let's cover each in turn.

Infrastructure for malleability

We've discussed how most of the *infrastructure* used to run and develop software today is designed without malleability as a priority. We've also shown some inspirations for reorienting around malleability. The desktop filesystem enables shared data between tools. Smalltalk, Hypercard, and spreadsheets enable live modification by allowing code editing at runtime. Browser-based apps encoding their UIs using the DOM enables permissionless modding through extensions.

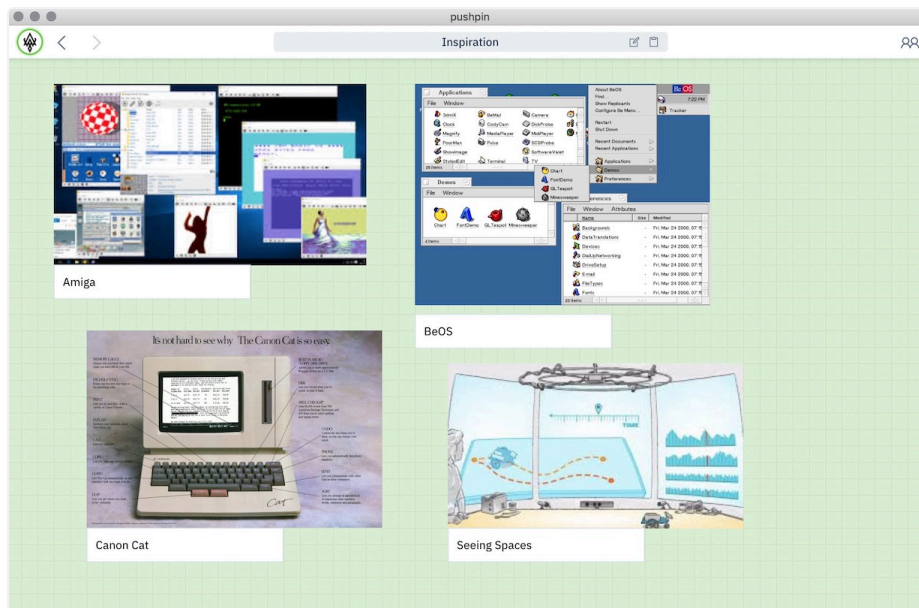
Inspired by these predecessors, **we have been prototyping an infrastructure stack for building, running, and sharing software that prioritizes malleability**. Our approach builds on another area of our research: local-first software, which is a philosophy that prioritizes giving users ownership and control over their data, while maintaining the ability to collaborate with others. As part of that work, we developed a collaboration library called Automerger, which persists and synchronizes JSON documents among users.

In fact, one reason we originally worked on local-first software was that we thought it could provide an important ingredient for supporting malleability.

PushPin: an extensible media canvas

To start figuring out how a local-first foundation could enable malleability, we prototyped PushPin, a web-based collaborative media canvas. A key idea we developed in that project was “document functional reactive programming” (DFRP): representing a tool as a UI component authored in React, backed by a JSON document that was automatically persisted and synchronized through Automerge.

DFRP made it less work to extend the interface than in traditional applications, since we could just add a UI component without worrying about backend databases or REST APIs. Furthermore, the decoupling of data and UI made it straightforward to define new user interfaces for *existing documents*—since we could define a new component and then register it as an alternate editor.



A PushPin board with embedded cards, each implemented as a React component backed by its own synced Automerge document

PushPin also showcased several challenges. First: how do you choose which UI to use to show a document in a given context? We tried a simple model based on hard-coded contexts—“show a compact view when rendering the document as a card on a board; show an expanded view when rendering as the main content”—but this approach wasn’t sufficient for all our needs. For one thing, it didn’t allow users to choose among views depending on their task.

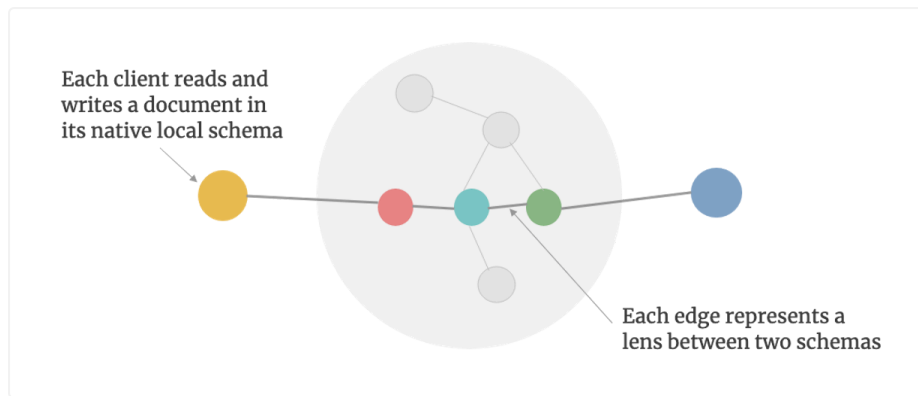
We also found that an embedding model needs to balance isolation and coordination. We used a restrictive model in which components can embed one another but no further communication was allowed. While this strong isolation made it easy to reason about behavior within a given component, there was no easy way for UI components to share context like hover states, which prevented certain rich user experiences. And permissionless modding wasn’t well supported either, since there was no mechanism to reach into an existing component. These questions of dispatching views and coordinating across components remain open areas of exploration for us.

Cambria: mediating schema compatibility across tools

Schema compatibility was a point of frustration using PushPin. Upgrading tools would frequently break compatibility with existing data. And it was difficult to make different tools interoperate if they couldn’t agree exactly on the format of their underlying JSON data. This is an instance of a broader class of challenges—how should tools cooperate if they can’t agree on the shape of their data? In a malleable environment, we can’t rely on a central authority to dictate the schema.

We prototyped a system called Cambria which solves this challenge by decoupling write schemas from read schemas. Updates are written by each tool in its preferred schema, and then interpreted on demand in other schemas—which could include newer/older versions of the schema, as well as entirely different schemas used by other tools. These are not one-time data migrations; they are ongoing live translations, preserving the ability to write and read from any schema simultaneously.

Based on some limited testing of the Cambria prototype, we believe live data translations are a promising direction for supporting pluralistic schemas. However, we have not yet built a production-ready version of this system that integrates with Automerge; one challenge is that it may require deep integration with the underlying data engine.



Cambria supports translating data between tools live on demand, through a graph of transformations called *lenses*.

Farm: code is data

Although PushPin lowered the bar to contributing new tools to the environment, the system wasn't editable at runtime—the code was stored in a GitHub repository, and deployed through a standard web application pipeline. Thus, in our next project, we experimented with enabling fluid modifications by **using the local-first data layer to host not just user data, but also the code for the system itself**. We built a system called Farm which allowed users to author tools in the Elm language, with the source code stored and synced in Automerge documents.

We chose Elm because stricter compilation rules helped automate decisions for when to synchronize code with collaborators.

At a basic level, treating code as data streamlined the sharing of tools. However, many aspects of collaboration proved challenging. How could you work on a tool without breaking it for other people as you edited it? And how could multiple collaborators work on different aspects of the same tool in parallel? This set of challenges led us to our next project.

Patchwork: version control + bootstrapping

Enabling exploration and divergence lies at the heart of malleability. Different users need the freedom to try changing a piece of software or make multiple variations, without fear of losing work or messing up their collaborators. In traditional software engineering projects, these problems are solved with version control concepts, like Git branches.

While version control systems are useful tools for software engineers, they're not designed to support malleable software. They are difficult to learn, and targeted at formal collaboration workflows. We believe that malleable software demands new solutions which prioritize accessibility to a broad set of users, and which better support casual, informal exploration. We call this vision universal version control.

To explore these version control ideas further, we created Patchwork—a web-based collaboration environment for malleable software. Patchwork builds on the core ideas of PushPin and Farm, storing both user data and software code in Automerge documents. On top of that, it adds version control utilities like history views and simple branching. These tools apply to any document in the system—whether a piece of writing, or code for a software tool.

oose to go to the Moon

me say, go to the Moon? Why choose this as our goal? And they k, why climb the highest mountain? Why, over 100 years Atlantic? We choose to go to the Moon.

o go to the Moon in this decade and do the other things, not y are easy, but because they are hard; because that goal will nize and measure the best of our energies and skills because

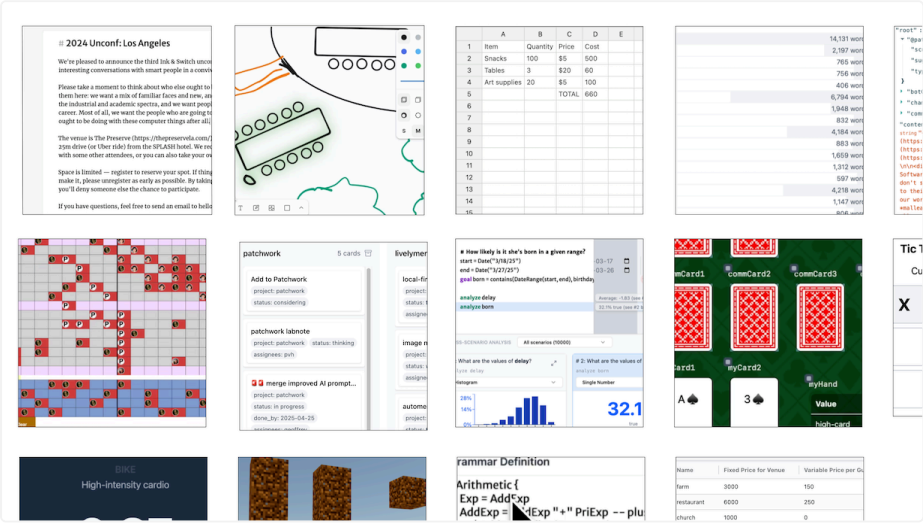
go to

35 → over 100

Why does Rice play Texas?

Patchwork can display diffs on branches of text documents, among other media types.

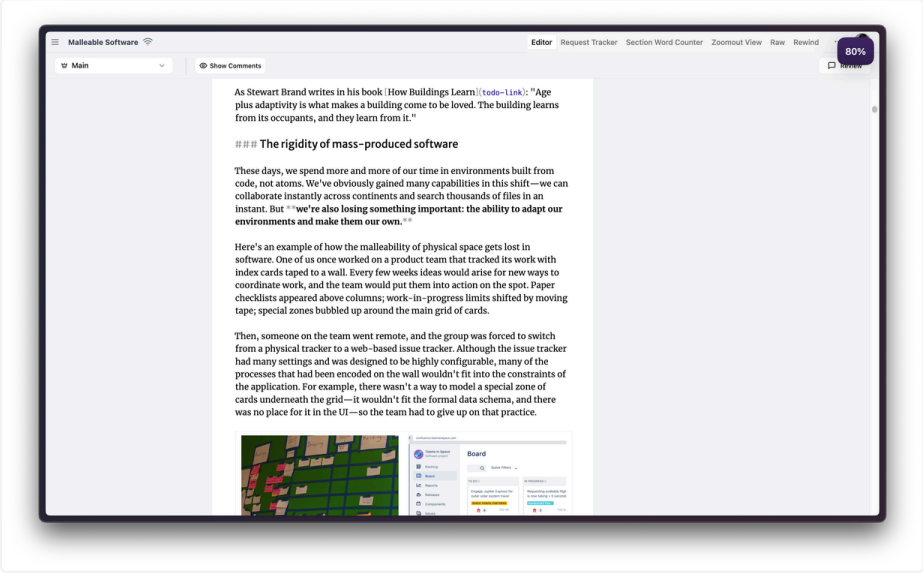
One process difference between Patchwork and our previous infrastructure projects is that we are now “bootstrapping”—most of our lab’s internal knowledge work, from writing to whiteboarding to project planning, happens inside of Patchwork. In fact, these very words are being typed into a collaboratively edited Markdown document hosted in Patchwork.



We’ve used a broad variety of tools hosted in Patchwork—from text editors and whiteboards to simulations, audio sequencers and games.

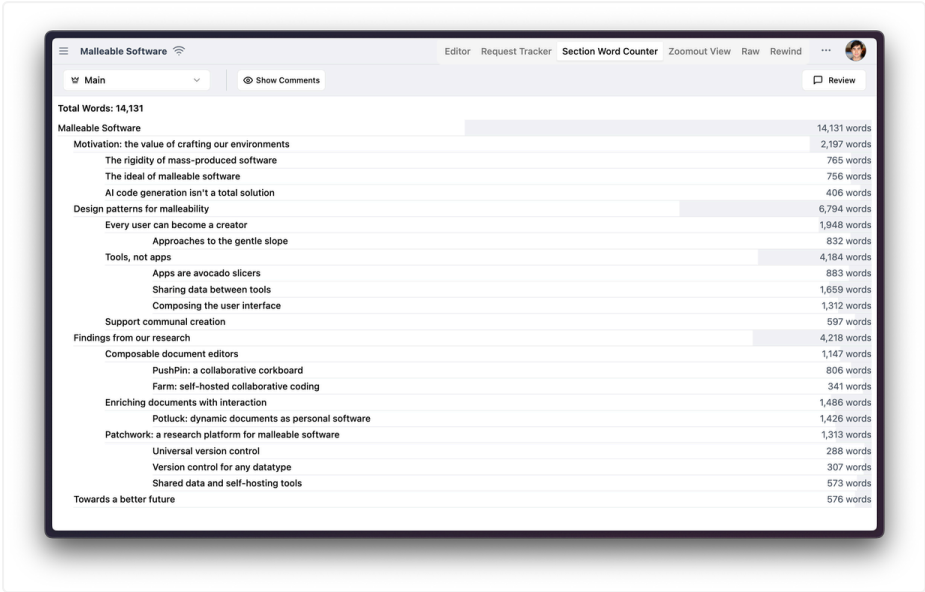
There’s a long tradition of bootstrapping in computing research project, dating back to Doug Engelbart’s NLS project. For more of a history see [this talk](#) by Bret Victor.

One thing we’ve observed from using Patchwork for our work is that opportunities for improving your software can emerge naturally within a malleable system. Here’s an example from the process of writing this essay. The main tool we use is a collaborative Markdown editor:

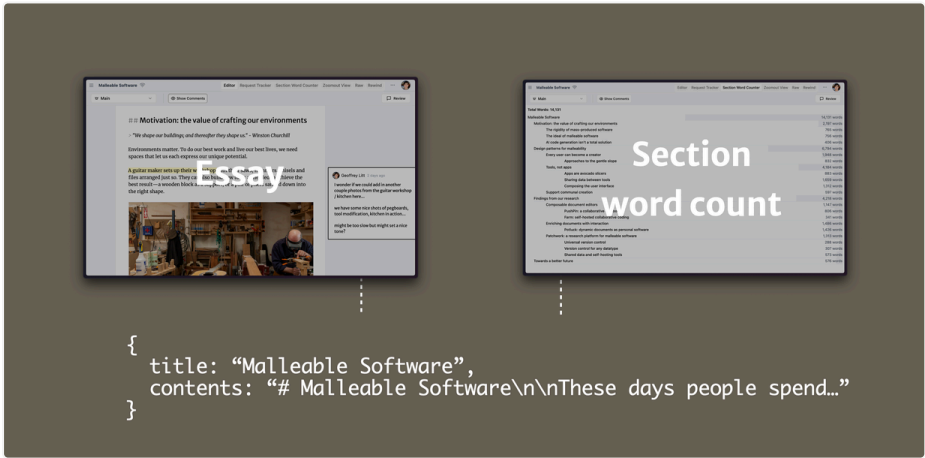


Editing this essay in Patchwork

At some point, we realized the essay was too long, but it was hard to see the length of different sections just by looking in the Markdown editor. So we created a minimal Section Word Counter tool that measured the length of each section:

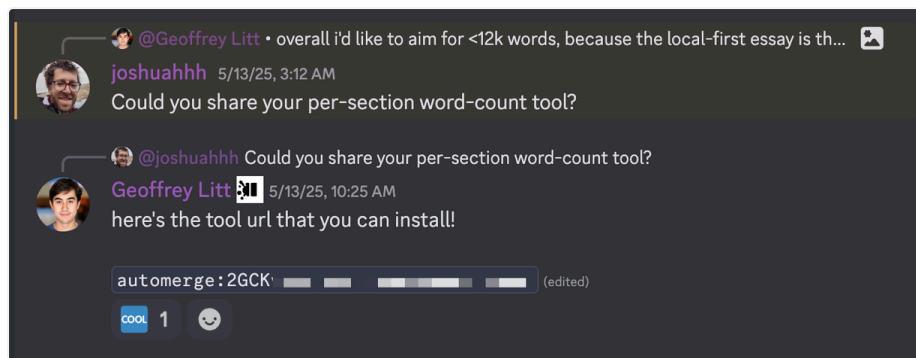


In Patchwork, this new tool could be straightforwardly integrated into the existing workflow. It was coded in a desktop IDE and then synchronized into an Automerge document, with no need to rebuild or redeploy Patchwork itself. Once installed, the Section Word Counter could instantly be used to view the existing essay document, with live updates as the essay changed.



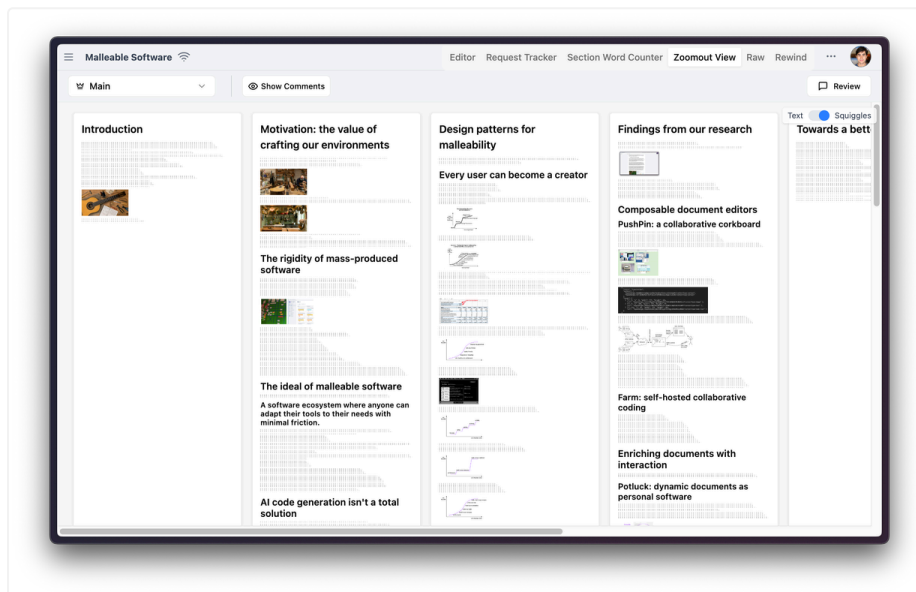
Two Patchwork tools can edit/view the same underlying Automerge document.

Treating code as data also simplifies sharing. The word counter tool was initially developed by one of the authors for his own personal use, and only loaded in his own Patchwork instance. But when another author asked to use the tool, it could be shared with a link, just like any other document. Each collaborator could choose which tools they wanted to install based on their own needs.



Sharing a tool with a collaborator via URL. (Chat is one use case we haven't managed to migrate into Patchwork yet)

We used a similar approach to develop several other tools as needs arose. A “Zoomout View” proved useful for talking about the overall structure of the essay:

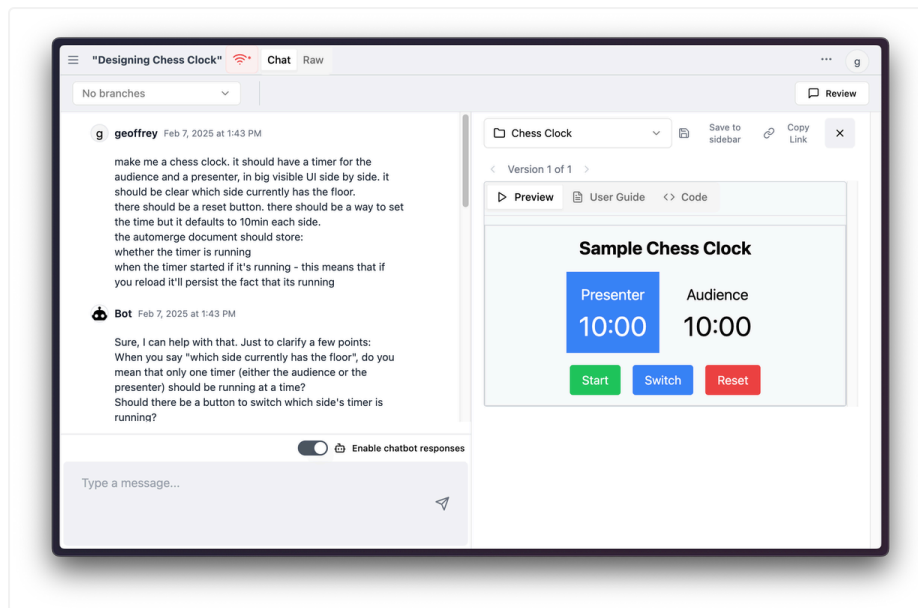


In some cases, new tools have proven less useful. We built a “Request Tracker” that would manage assigning feedback requests to coauthors, but this tool didn’t end up getting used very much. That wasn’t a huge loss, though—it didn’t take much work to create the tool, and it was easy to ignore and uninstall.

Another thing we’ve found while customizing Patchwork is that **AI is a useful complement to a malleable environment**. We argued earlier that AI-assisted coding alone does not guarantee malleability. But *when combined with a malleable environment*, AI-assisted development can make it much faster to edit your tools.

We’ve used AI assistance to rapidly build many tools in Patchwork. The Section Word Counter tool mentioned above was coded with AI assistance in just a few minutes—in the middle of a writing session, without needing to set aside dedicated time.

A malleable environment can also provide platform capabilities that make AI-generated software more useful. For example: we have an interface for making small software tools from an AI chat. While this UI superficially resembles existing products like Claude Artifacts, the generated tools gain capabilities from existing inside of Patchwork. They automatically support persistence and multi-user collaboration, and can compose with existing tools for editing existing data.



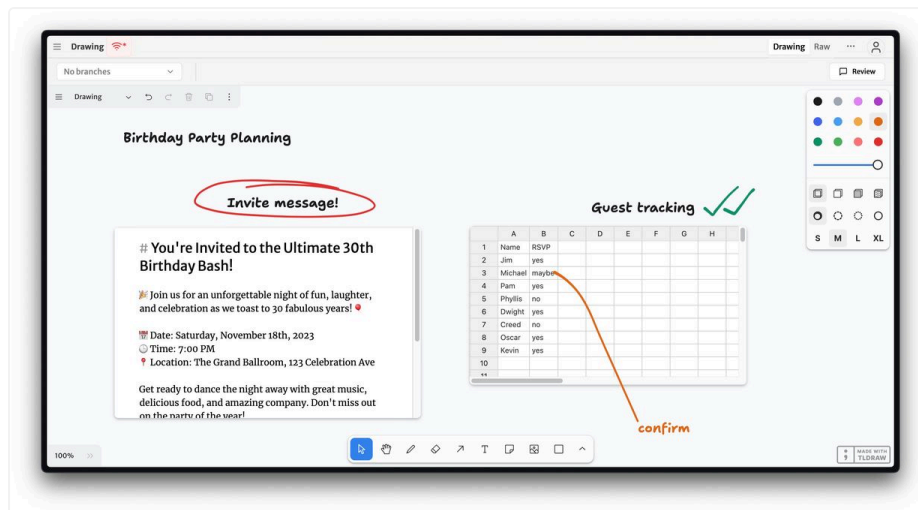
A presentation timer tool built with an AI chat in Patchwork has automatic persistence and multi-user synchronization.

While we've already experienced some glimpses of malleability in Patchwork, some big challenges remain.

One open challenge is fully applying the ideas of lightweight universal version control to code. Although we have a basic mechanism for creating a branch of a software tool, we've yet to flesh out a full set of capabilities around branching—such as seeing and running different branches of a tool simultaneously. And more broadly, we've noticed there are tricky social challenges around collaborating on software in a less centralized way. How do people negotiate whether to share the same version of a tool or go their own way with separate forks? Who makes product decisions for a given tool—does the original creator stay in charge, or do they allow anyone to freely contribute? While these questions can't be magically resolved, we think versioning tools can help.

A related challenge is managing expectations of quality. Most of the tools we've built aren't anywhere close to the polish level of commercial products; they're scrappy personal tools. When someone shares a tool, how can they communicate its level of quality? There's a difference between sharing a tool "as-is" and committing to ongoing maintenance.

Finally, Patchwork currently has a limited embedding model that we'd like to improve on. Tools can embed other tools, but the UI of the embedded tool gets cramped, and users need better ways to choose tools at all levels of the hierarchy. We've also gradually discovered many reasons why UI components need to communicate in richer ways. For instance, when working on a generic commenting utility that could work with any tool, we realized that it's useful for tools to report the current user selection as a semantic region of a document. We've added hardcoded workarounds to enable these kinds of tool composition, but we'd like to find a more general and less coupled approach, perhaps inspired by approaches like entanglers and Varv we mentioned earlier.



Embedding a text document and a spreadsheet inside a whiteboard in Patchwork

Patchwork is still an active project. We've published some early findings from our version control work in a [lab notebook](#), and we plan to share more about the broader system as we make progress. Eventually we also plan to release Patchwork as an open-source tool.

Dynamic documents

In our infrastructure work, we've innovated on how software is hosted, but the behavior itself has been specified with traditional code. And yet, we know that writing code—even with AI assistance—still represents a steep “cliff” between usage and modification.

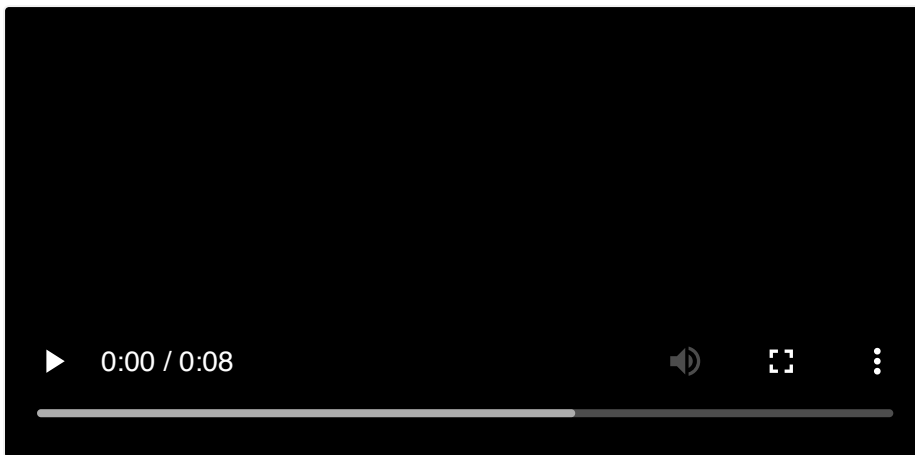
Thus, **another strand of our work has focused on allowing the creation of custom tools without programming.** Above, we showed some prior art like spreadsheets and Hypercard which embodied the approach of starting with some directly manipulable information and optionally layering on programming.

We've followed in this tradition in our work on **dynamic documents**. Our goal is to explore new techniques for letting users layer behavior onto their existing media documents. We've aimed to figure out: how should users represent information to make it possible to enrich with interaction? And how can tools cooperate and coexist within a document as a shared fabric?

Potluck: dynamic plaintext for recipe planning

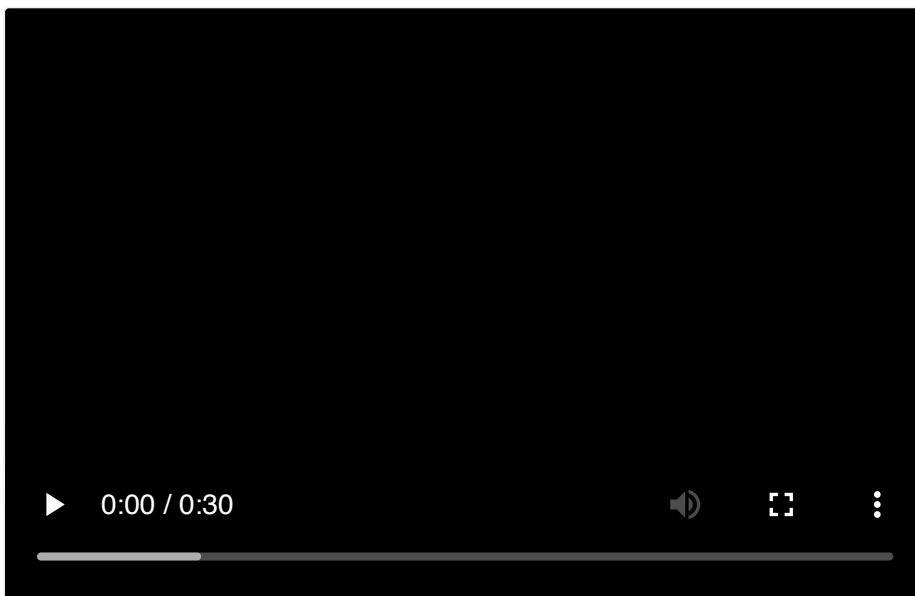
In [Potluck](#), we explored how users might enrich text notes with dynamic behavior like scaling ingredients and setting timers in a recipe. Users could represent information with any text syntax they found natural, and then write detectors that would parse meaningful structure out of the text.

We found that Potluck could support the creation of many useful tools, and it felt nice to jot down information in any format that felt natural.



Scaling a recipe in Potluck document. The behavior of the scaling slider is specified within the Potluck editor using a formula language.

AI assistance also integrates nicely into Potluck. The AI can draft a set of detectors and computation rules, while preserving the user's ability to see and edit the generated logic themselves in a live programming environment.



Doubling quantities in a recipe with AI support. The generated detectors and formulas can be directly viewed and edited.

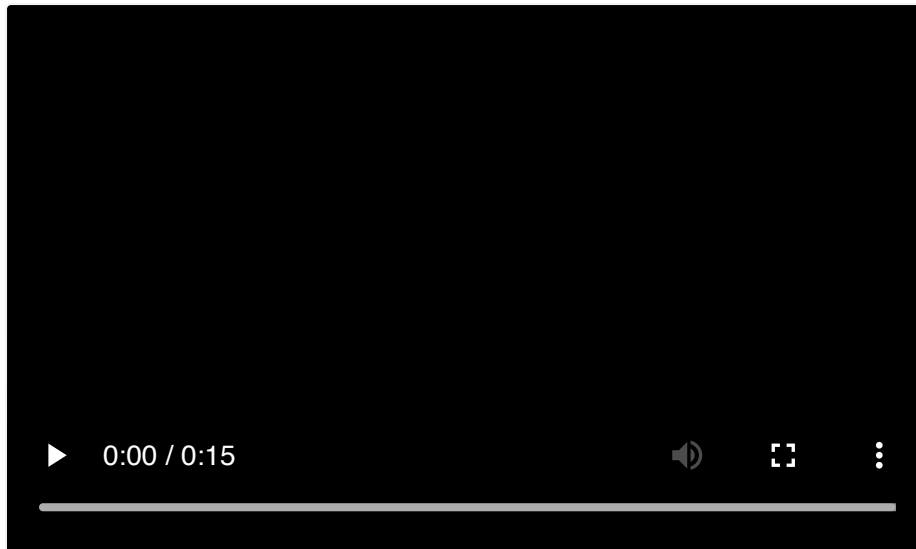
However, one problem with Potluck was that parsing structure from arbitrary plaintext proved cumbersome. We struggled to represent connections between related information (like containment or sequence) without resorting to complex detector rules.

Embark: dynamic outlines for travel planning

In our next project, we decided to try a more structured approach. We built an editor for travel planning documents called Embark, which used a hierarchical outline as the base format. The outline provided some low-level structure that resolved some of the issues with detecting complex patterns in Potluck.

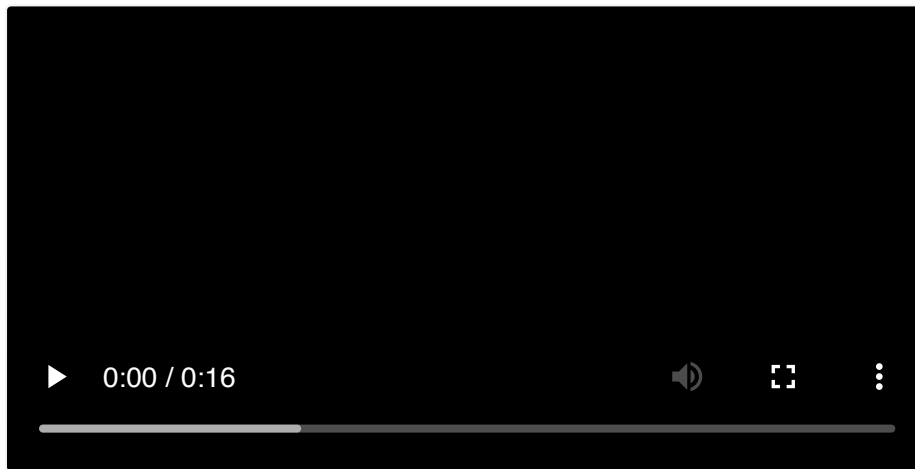
We also added a first-class concept of structured information. An object like a Google Maps location could be “mentioned” and stored within the outline. These objects could then be used as inputs to computations like routing and weather forecasting.

Finally, we allowed rich views like maps and calendars to be embedded in the outline, to visualize and interact with information about the trip.



Computing a driving route in an Embark document. The route calculation draws on context from the surrounding document.

A key takeaway from Embark was that **an embedding model with rich shared context enables powerful interactions**. The embedded views in Embark are deeply aware of surrounding information—a map view can read and visualize the locations in the outline. The available information depends on *local* context—you can limit what information is shown on a map by moving it to a different part of the outline. Interactions are also synchronized—hovering on a place in the map highlights it in the outline, and vice versa. The cumulative result of these mechanisms is a cohesive user experience that’s not available in a more restricted embedding scheme, like the one we have in Patchwork currently, or one website embedding another one as an `<iframe>`.



In Embark, a map can interact with structured data in the outline document.

A major challenge we faced in both Potluck and Embark is navigating the tension between more and less formal representations of information. It’s valuable to be able to express your thoughts in an unstructured way, but computational tools typically benefit from operating on more structured representations. Furthermore, data schemas aren’t limited to user input—Google Maps locations have their own structured format that needs to be usable across various tools.

In our research on Programmable Ink, we’ve explored gradual enrichment over very unstructured data: sketchy drawings.

Finally, it remains to be seen whether the ideas of Potluck and Embark can be generalized further to handle a broader variety of tasks. We grounded each of those projects in the specific use cases of recipes and travel to limit our scope and encourage

authentic use, but we plan to try generalizing these ideas further and integrating them on top of a malleable infrastructure foundation such as Patchwork.

Towards a better future



Over the course of this essay, we've laid out a vision of a different kind of computing – one that could empower people to craft their own tools, shift power towards local communities, and help people do their best work. We've also shared many ideas for how exactly this different kind of computing might come about, from prior art and our own work.

At the same time, we also don't want to understate the challenges that lie ahead. Enormous effort over many decades has gone into refining application-centric computing, and forging a new path will require grappling with many difficult questions. We've covered some of the questions we see as fundamental: How can we create gentle slopes from passive use to active creation? How can we break up applications into composable tools? How can we keep the needs of collaborating groups in focus?

That is still a partial list, and there are big challenges we've mostly left out of scope in this piece.

Privacy and security: How do we reconcile the desire for extensible software and interoperability with the reality of bad actors? When untrusted strangers are sharing modifications to existing software that can access sensitive data, dangerous things can happen.

So far, in our own work we've focused on collaboration within trusted groups like our coworkers, but we suspect that new computing models may be needed to expand outside that circle of trust. See [Bernhard Seefeld's work](#) for one inspiration.

Business models: How would developers make money from their software if they were shipping composable tools, not monolithic applications? How is support and maintenance paid for?

Culture: How do we cultivate a movement towards personal agency where people *want* to modify their environments, both digital and otherwise?

These are daunting challenges. Technical capabilities can't be a full solution; economic and cultural shifts will also be required. But change is possible—computing is still young, it has changed a lot in the past decades, and surely many structural shifts still lie ahead.

Many different kinds of people can play a role. If you are a computing researcher, it's a ripe time to reimagine the foundational metaphors of computing to put more

power in users' hands. If you are a platform developer, consider tipping the balance towards seeing end-users as capable participants rather than passive users that need to be protected at all costs. Security and ease of initial use are virtues, but extensibility and power are as well. And if you make software products, consider how you might empower your users to take matters more into their own hands, rather than dictating every decision. While we believe radical change is needed to fully realize malleable software, incremental improvements are helpful too.

When the people living or working in a space gradually evolve their tools to meet their needs, the result is a special kind of quality. While malleable software may lack the design consistency of artifacts crafted behind closed doors in Palo Alto, we find that over time it develops the kind of charm of an old house. It bears witness to past uses and carries traces of its past decisions, even as it evolves to meet the needs of the day.

Everyone deserves the right to evolve their digital environments. It's an important way to fulfill our creative potential and maintain a sense of agency in a world that is increasingly defined in code. We hope you'll join us in making malleable software a reality.

To follow our ongoing research, subscribe to our [email newsletter](#).

We are grateful to:

- *Roshan Choxi, Orion Henry, Ignatius Gilfedder, Mark McGranaghan, Alexander Obenauer, Jeff Peterson, Max Schoening, Paul Shen, Paul Sonnnentag, Matt Tognetti, and Adam Wiggins, for contributions to the research projects mentioned in this essay.*
- *Mary Rose Cook, Marcel Goethals, Alex Komoroske, Clemens Klokmoose, Steve Krouse, Jess Martin, Andy Matuschak, Paul Sonnentag, Alex Warth, and Adam Wiggins for valuable feedback on the essay.*
- *Thomas Mitchell Clark, Guitar Builder; and Nathan Peterson, Food Practitioner; for allowing us to photograph their creative spaces.*